Beágyazottszoftver-fejlesztés

6. gyakorlat tematikája

A mostani gyakorlat során a STK3700 kártya BSP-jével (Board Support Package) ismerkedünk meg, ami a házifeladat elkészítése során is segítségünkre lesz.

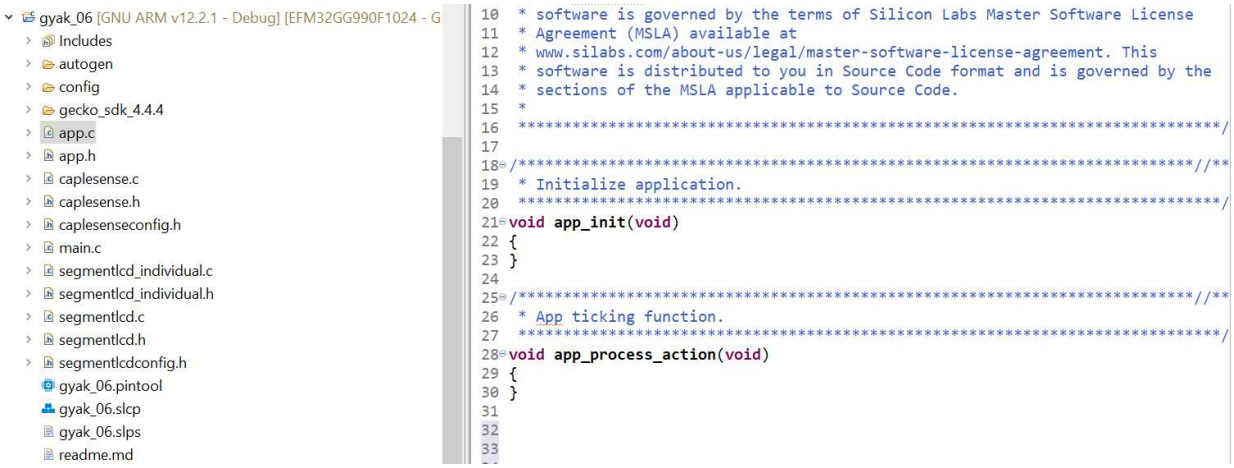
Sajnos tisztán a Simplicity Studio konfigurátora segítségével nem lehet összeállítani egy teljes és működő támogatást az STK3700 kártyára (vélhetően ezekkel a funkciókkal csak a legújabb kártyákat támogatják), ezért a kártya támogatás összeállításánál több lépés végrehajtására, és némi fájl másolásra is szükség lesz. Ezek a lépések egyébként nagyrészt hasonlítanak ahhoz, mint amikor egy a neten elérhető szenzor, vagy más külső IC támogatását szeretnénk a projectünkbe integrálni.

1. STK3700 BSP integrálása a projectünkbe

Hozzunk létre egy üres projektet (Empty C Project), majd másoljuk a honlapon található touchslider_plus_LCDindividual_for_SSv5.zip (Egyetemi hálózat, vagy bejelentkezés a tanszéki honlapra szükséges lehet) fájl tartalmát az app.c, app.h, valamint a touchslider_LCD_hasznalat.txt kivételével a project főkönyvtárába. Az app.c, app.h fájlok a letölthető zip fájlban egy demó mintaprogramot tartalmaz, későbbiekben ezt is érdemes megnézni. A jelenlegi project is tartalmaz egy app.c, app.h file-t, de ezek jelenleg üresek, az eredeti project struktúra alapján a felhasználói alkalmazást tartalmazzák, annak inicializációs, és periodikusan hívott főfüggvényével. Ezen a gyakorlaton ezt a fájt fogjuk kiegészíteni saját kódunkkal, így nem a main()-ből indulunk ki direktben.

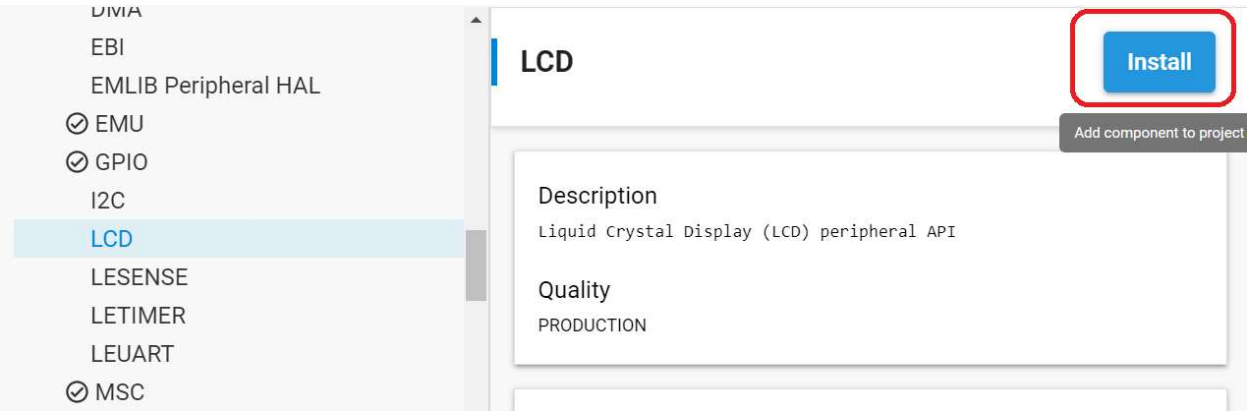
(https://www.mit.bme.hu/system/files/oktatas/targyak/12267/touchslider_plus_LCDindividual_for_SSv5.zip)

A projectünknek ehhez hasonlóan kell kinéznie:



Függőségek hozzáadása a projecthez.

Az emlib LCD kezelős szoftver komponensének (em_lcd.c, em_lcd.h) hozzáadása .slcp fájl segítségével az eddigiekhez hasonló módon: Platform / Peripheral / LCD



2. A szegmenses LCD kipróbálása

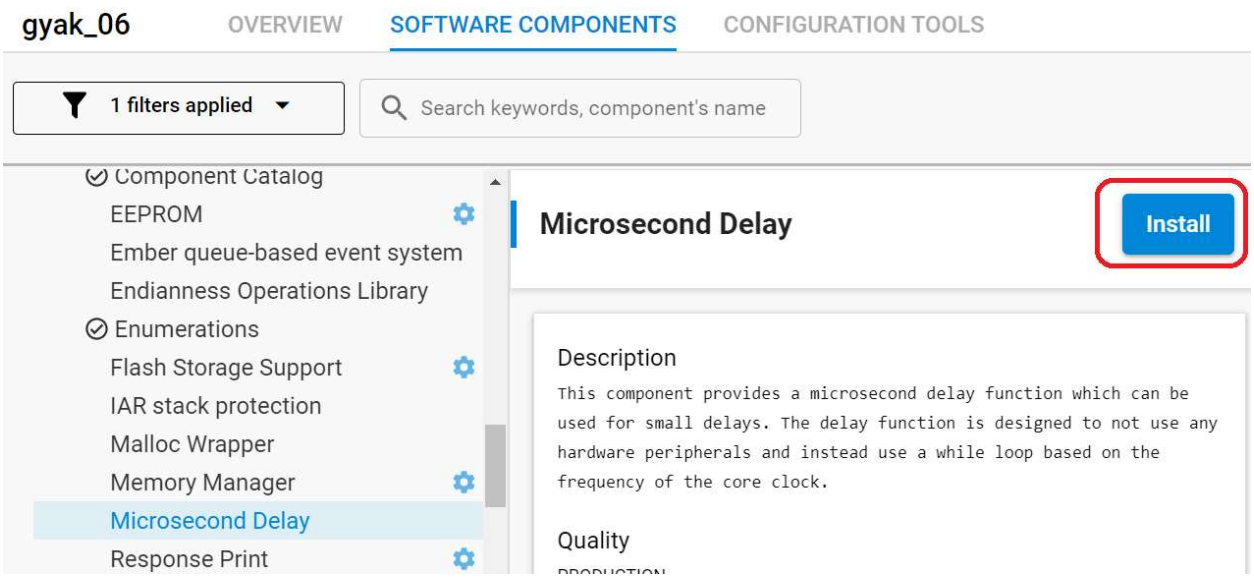
Próbáljuk ki a szegmenses LCD-t. Ehhez első lépésként a **SegmentLCD_Init()** függvényt kell meghívni. Majd írjuk ki egy **hello üzenetet** a szegmenses LCD-re a **SegmentLCD_Write**(**const char** *string) függvény segítségével. A szegmenses lcd egy speciális LCD típus, ami kis LCD lapokból rakja össze a megjelenítendő információt. Ez komplexebb megjelenítést tesz lehetővé, mint a hagyományos 7 szegmenses kijelzők, de kevésbé rugalmas, mint a divatos pixel alapú LCD kijelzők. A szegmenses LCD-k, bár kevésbé rugalmas a viselkedésük, mint a pixel alapú LCD-ké, de jóval fogyasztás takarékosabbak, és olcsóbbak, ezért számtalan nagy darabszámú alkalmazásban használják őket, mint például áram, gáz mérőórák stb.

Az **app.c** fájl így módosult az új részlet hozzáadásával.

```
/**************************************************************************//**
 * Initialize application.
 *****************************************************************************/
#include "segmentlcd.h"

void app_init(void)
{
  /* Enable LCD without voltage boost */
  SegmentLCD_Init(false);
  SegmentLCD_Write("Hello");
}
```

A következő feladatokhoz érdemes egy késleltetést okozó függvényt hozzárakni. Ehhez a Silabs által adott késleltető függvényt fogjuk használni. Ezt hozzá kell adnunk a projectünkhöz a Platform / Utilities / Microsecond Delay útvonalon a szokásos .slcp fájl segítségével.



Vigyázzunk, ez egy blokkoló késleltetés, ami lekérdezi a CPU működési frekvenciáját, és az alapján végrehajt a késleltetésnek megfelelő számú utasítást (kiszámított for ciklus gyakorlatilag). A késleltetés függvénye **sl_udelay_wait** aminek a paramétere a késleltetendő us-ok száma.

Készítsünk egy demó alkalmazást, ami használja a szegmenses LCD más mezőit is. Az alábbi függvényekkel:

void SegmentLCD_LowerNumber(int num);

void SegmentLCD_Number(int value);

void SegmentLCD_Symbol(lcdSymbol s, int on);

Az **app.c** fájl így módosult egy lehetséges demó hozzáadásával:

```
/***************************************************************************//**
 * Initialize application.
 ******************************************************************************/
#include "segmentlcd.h"
#include <sl_udelay.h> // blocking delay function

int32_t app_upper_number = -999;
int32_t app_lower_number = -999;
lcdSymbol app_lcd_symbol = LCD_SYMBOL_GECKO;
bool app_lcd_symbol_on = true;

void app_init(void)
{
  /* Enable LCD without voltage boost */
  SegmentLCD_Init(false);
  SegmentLCD_Write("Hello");
  sl_udelay_wait(1000000);
}

/***************************************************************************//**
 * App ticking function.
```

```
 *****************************************************************************/
void app_process_action(void)
{
  SegmentLCD_LowerNumber(app_lower_number);
  app_lower_number+= 11;
  SegmentLCD_Number(app_upper_number++);
  SegmentLCD_Symbol(app_lcd_symbol, app_lcd_symbol_on);
  if(app_lcd_symbol == LCD_SYMBOL_DP10)
    {
      app_lcd_symbol = LCD_SYMBOL_GECKO;
      app_lcd_symbol_on = !app_lcd_symbol_on;
    }
  else
    {
      app_lcd_symbol++;
    }

  sl_udelay_wait(200000);
}
```

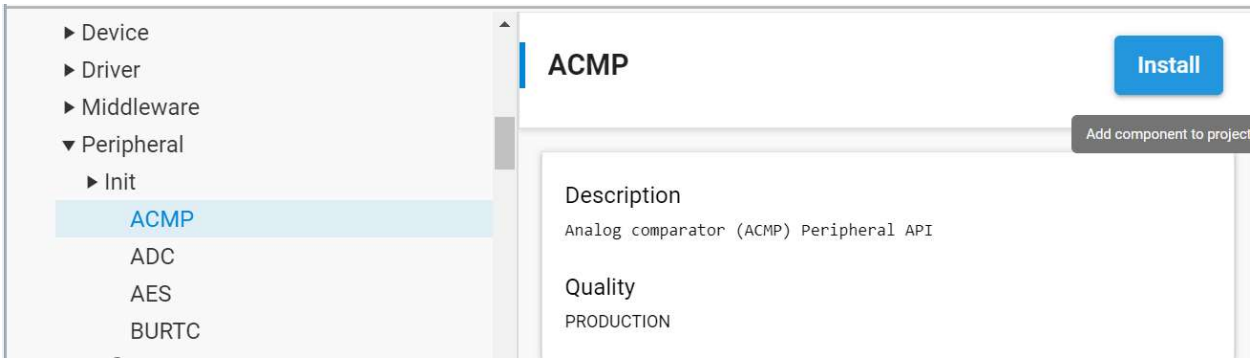
A szegmenses lcd alacsonyabb szintű kezeléséhez, ami sok házi feladatban szükséges a Naszály Gábor által írt **segmentlcd_individual** fájlok függvényei nyújtanak segítséget, ezek működését az eredeti .zip fájl app.c fájlában lévő demó szemlélteti.

3. Touch Slider kipróbálása

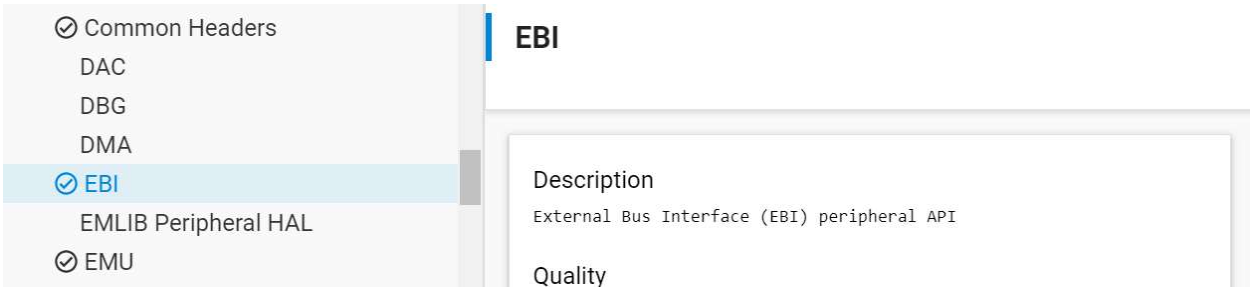
Az STK3700 kártya egy érdekes perifériája a kapacitív úton működő TouchSlider. Ennek használata is sok házifeladatban szükséges, ezért nézzünk erre is egy demót.

A TouchSlider rengeteg függőséggel rendelkezik, ezeket egyesével hozzá kell adnunk a projecthez:

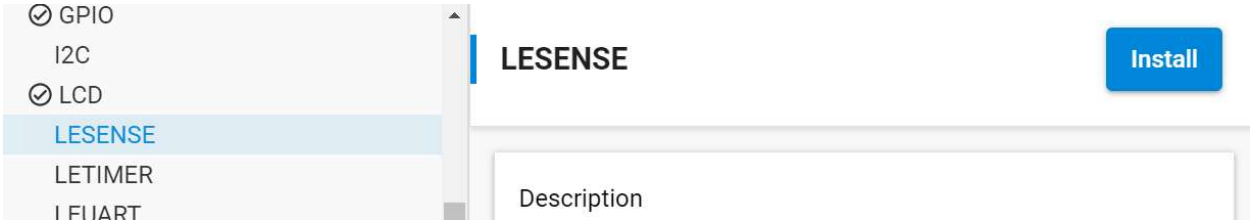
Platform/Pheripheral/ACMP analóg comparator.



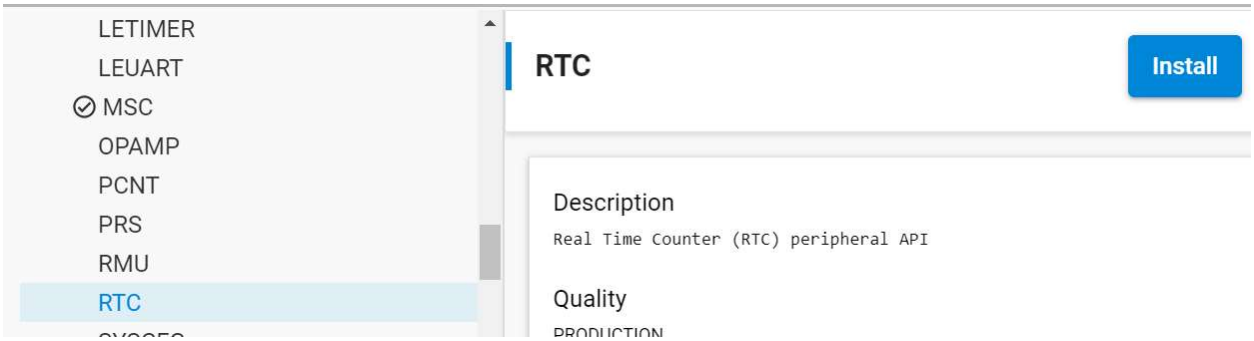
Platform/Pheripheral/EBI External bus interface



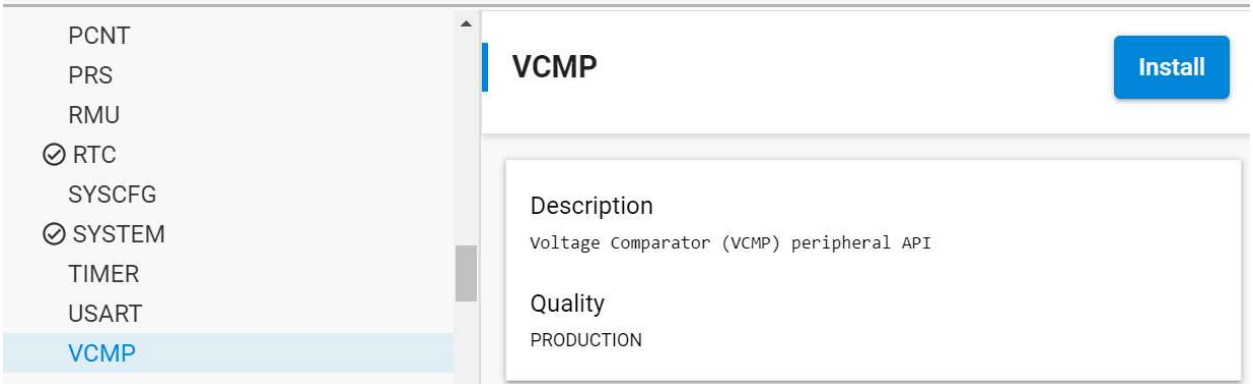
Platform/Pheripheral/LESENSE alacsony energiájú mérés támogatás



Platform/Pheripheral/RTC



Platform/Pheripheral/VCMP



Inicializáljuk a touch Slidert a **caplesense.h** fájlban található függvénnyel **CAPLESENSE_Init** jelenleg nem akarunk energiatakarékos működést kérni az eszköztől, így a sleep parméternek false értéket adhatunk.

A kapacitív érzékelő pozícióját a int32_t **CAPLESENSE_getSliderPosition**(**void**) függvénnyel le lehet kérdezni. Jelezzük ki ezt a pozíciót a szegmenses LCD-re.

Az **app.c** fájl így módosult egy lehetséges demó hozzáadásával:

```
/***************************************************************************//**
 * Initialize application.
 ******************************************************************************/
#include "segmentlcd.h"
#include "caplesense.h"

#include <sl_udelay.h> // blocking delay function

int32_t app_upper_number = -999;
int32_t app_lower_number = -999;
lcdSymbol app_lcd_symbol = LCD_SYMBOL_GECKO;
bool app_lcd_symbol_on = true;

void app_init(void)
{
  /* Enable LCD without voltage boost */
  SegmentLCD_Init(false);
  SegmentLCD_Write("Hello");
  CAPLESENSE_Init(false);
  sl_udelay_wait(1000000);
}

/***************************************************************************//**
 * App ticking function.
 ******************************************************************************/
void app_process_action(void)
{
  SegmentLCD_LowerNumber(CAPLESENSE_getSliderPosition());
  sl_udelay_wait(1000);
}
```